A SYMBOLIC MUSIC BENCHMARK FOR MUSIC THEORY REASONING AND COMPUTATIONAL MUSICOLOGY

Anonymous Authors
Anonymous Affiliations
anonymous@ismir.net

ABSTRACT

We present a symbolic benchmark for evaluating whether large language models can identify, transform, and explain core music theoretical structures under reproducible input output contracts. The benchmark defines eight zero-shot tasks in two families: label classification and note-sequence transformation. Cases are generated by deterministic rule-based procedures, exported into three synchronized tokenizer views (`note_level`, `midilike`, and `remilike`), and evaluated with automatic verifiers. The benchmark also supports an optional explanation layer for qualitative analysis of musicological reasoning, without changing the primary evaluation target. The current release is intentionally constrained as a V1 scope: harmonic function is limited to major mode, voice leading uses a two-voice setting, and chord vocabulary is reduced to a compact tonal subset. This benchmark is designed both as a controlled engineering artifact for comparing models and as a computational musicology resource for studying the gap between symbolic correctness and analytical explanation.

1. INTRODUCTION

Benchmarking has long played an important role in MIR research, from community-wide evaluations such as MIREX to more focused datasets and challenge tasks [1]. Recent large language model systems can generate plausible symbolic music sequences, but generation quality alone does not establish competence in music theory. A model may produce fluent token sequences while still failing on interval naming, harmonic function assignment, or deterministic melodic transformation.

This paper targets that gap with a benchmark centered on a simple question: *can language models correctly capture and manipulate core music theory structures, and does structural correctness align with meaningful analytical reasoning?* Instead of scoring open-ended musicality, we ask whether a model can produce verifiable outputs for tasks corresponding to standard analytical or transformational concepts. Compared with broader music understanding evaluations [2], we focus on a narrower but cleaner task set where scoring can remain fully automatic and auditable.

Our design priorities are: (1) deterministic ground truth, (2) tokenizer-agnostic case semantics, (3) strict machine-checkable outputs, and (4) reproducible end-to-end execution. We contribute: (1) an eight-task symbolic benchmark that separates label prediction from note-sequence transformation; (2) a deterministic generation and verification pipeline with synchronized tokenizer views; (3) a standardized prompt construction scheme for local and API runners; and (4) an optional explanation layer that supports musicological analysis beyond strict correctness.

2. RELATED WORK

Symbolic music modeling has progressed through event-based tokenization and pretraining approaches, including REMI and MusicBERT [3, 4]. However, common evaluations emphasize continuation quality or downstream prediction rather than explicit theory-grounded reasoning. More broadly, language model evaluation research has emphasized benchmark design, structured outputs, and reproducibility [5]. Recent studies on language models for music understanding and generation evaluate broader capability sets and prompting behavior [2].

Our benchmark is complementary to that line of work. We reduce task breadth to improve determinism and reproducibility, while keeping the tasks close to recognizable analytical concepts from music theory. The benchmark also aligns with MIR's broader benchmark culture [1], but focuses specifically on symbolic theory tasks rather than audio classification, retrieval, or open-ended generation quality.

3. METHODOLOGY

3.1 Task Taxonomy

The benchmark defines eight tasks grouped by output type.

Label classification:

- `interval_identification`
- `chord_identification`
- `harmonic_function`
- `voice_leading`

Note-sequence transformation:

© Anonymous Authors. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Anonymous Authors, "A Symbolic Music Benchmark for Music Theory Reasoning and Computational Musicology", submitted to *ISMIR*, 2026.

- `transposition`
- `melodic_inversion`
- `retrograde`
- `rhythm_scale`

For label tasks, each case has one canonical target label. For sequence tasks, each case has one target event sequence.

3.2 Prediction Interface and Explanation Layer

The benchmark uses a canonical prediction interface. Label tasks return `prediction_label`; sequence tasks return `prediction_notes`; and an optional `prediction_explanation` field can be added for qualitative analytical output without affecting the primary metric. This separation lets the benchmark remain easy to run for non-expert users while preserving a second layer for paper-oriented analysis of explanation quality.

Figure 1 provides a compact visual summary of all eight tasks, grouped by output type and shown with input-rule-output structure.

3.3 Current V1 Scope

We intentionally frame the current release as a V1 scope rather than as full tonal coverage. Harmonic function is limited to major mode with the three coarse classes `tonic`, `predominant`, and `dominant`. Voice leading is simplified to a two-voice setting with labels `parallel_fifths`, `voice_crossing`, and `none`. Chord identification uses a reduced vocabulary focused on major, minor, diminished, augmented, and dominant seventh sonorities. These simplifications are methodological rather than theoretical claims about music itself.

3.4 Deterministic Case Generation

All cases are generated by rule-based scripts with fixed seeds. The generator creates canonical note-level payloads and deterministic targets, including event timing and duration fields for sequence tasks. Examples of deterministic operators include semitone transposition, inversion around pivot, order reversal, and duration scaling. For rhythm scaling, duration pools are chosen to keep outputs aligned with the benchmark grid.

3.5 Tokenizer Views

All cases are generated in a canonical event schema with fields `t` (start time), `p` (MIDI pitch), `d` (duration), and `v` (velocity). The export step then serializes the same payload into three synchronized tokenizer views:

- `note_level`: event blocks separated by `|`, e.g., `t=0.5 d=0.25 notes=[G4] v=80`
- `midilike`: repeating quintuplets, e.g., `TIME_0.5 NOTE_ON_67 VEL_80 DUR_0.25 NOTE_OFF_67`

Table 1. Task groups, outputs, and primary checks in the released benchmark.

Task group	Output field	Primary checks
Label classification	`prediction_label`	Accuracy, exact match, macro F1 where needed
Sequence transformation	`prediction_notes`	Exact match plus task-specific sequence metrics

- `remilike`: repeating quadruplets, e.g., `POS_0.5 PITCH_67 VEL_80 DUR_0.25`

For pitch-set inputs in label tasks, serialization is also tokenizer-specific: `note_level` uses bracketed note-name sets, `midilike` uses `CHORD_NOTE_*` tokens, and `remilike` uses pitch-class tokens. The exporter rewrites `input`, `target`, `system_prompt`, and `user_prompt` for each tokenizer while keeping `id`, `task`, `payload`, and `target_payload` aligned.

3.6 Prompt Design and Composition

Each benchmark item is prompted in a two-part format: a system instruction block and a user content block. The system block defines hard constraints such as required output type, no extra text, and tokenizer-specific output grammar. The user block is instance-specific and follows a fixed composition order: one task description line, condition or control lines when applicable, and serialized musical input rendered in one of the tokenizer views.

For label tasks, the expected output is exactly one label token. For sequence tasks, the expected output is exactly one transformed token sequence in the requested tokenizer format. We use deterministic line ordering so prompts are stable across runs and directly auditable in exported case files.

4. EXPERIMENTAL PROTOCOL

4.1 Execution Pipeline

The protocol is designed to isolate model behavior from data noise. We first generate canonical symbolic cases, then export tokenizer-specific views, and finally execute model inference on a unified case schema. Predictions are reattached to their source cases for item-level analysis. A regeneration policy controls when benchmark inputs are rebuilt, so repeated runs can be either strictly fresh or strictly comparable depending on the experiment goal.

4.2 Model Backends

The same benchmark cases are executed across local runtimes and API runtimes. We use one provider-normalized interface and explicit provider-prefixed model identifiers, which keeps model routing transparent while holding the

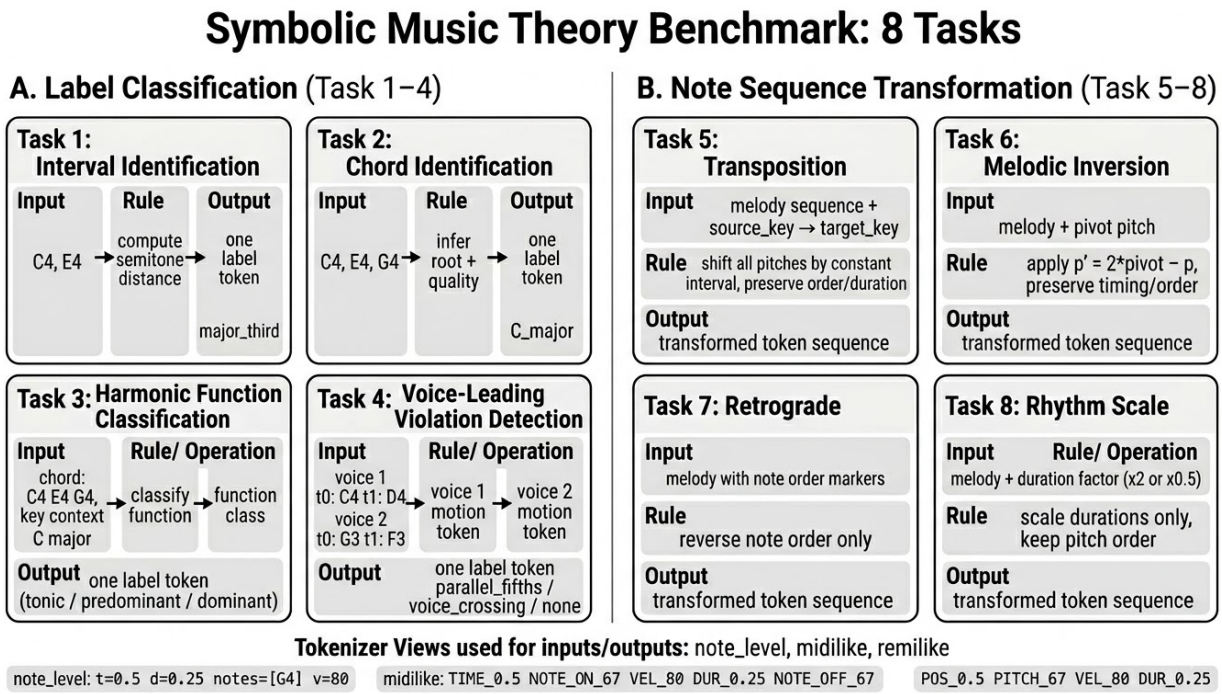

Figure 1. Benchmark task overview. Left: label-classification tasks. Right: note-sequence transformation tasks. Each panel shows the input form, core operation, and expected output type.

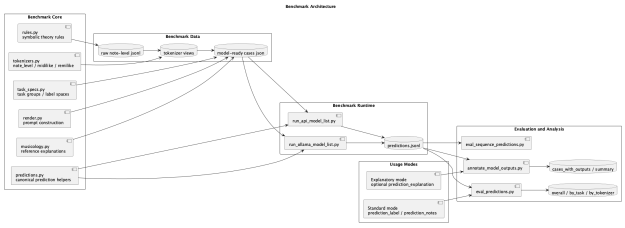

Figure 2. Benchmark architecture from generation and view export to model execution and scoring.

case format fixed. Each case is sent as an independent request, so no cross-item conversational state can influence scores.

4.3 Metrics

The primary score is binary hit or miss for all tasks. A prediction is a *hit* only when it exactly matches the ground truth label string or event sequence; otherwise it is a *miss*. No partial credit is used in the main benchmark score. Prediction files are post-processed into case-level annotations with match status. Optional diagnostics, including explanation fields, are retained for qualitative inspection but are not part of the primary score.

4.4 Artifacts and Reproducibility

The repository stores generated cases, raw predictions, annotated outputs, and summary reports so every reported number can be traced back to concrete inputs and outputs. This artifact chain supports deterministic reruns, error auditing, and direct comparison of model behavior under identical symbolic conditions.

5. RESULTS AND ANALYSIS

5.1 Pipeline Sanity Checks

Oracle evaluations achieve perfect scores by construction, confirming that generation, tokenizer rendering, and evaluators are internally consistent under gold predictions.

5.2 Pilot Model Behavior

Pilot runs show heterogeneous behavior across tasks and tokenizers. Label tasks are generally more stable than sequence transformation tasks, while strict token formatting can dominate failure modes in sequence outputs. This pattern is consistent with the benchmark's design goal of separating theory errors from format-compliance errors.

5.3 Task-Wise Accuracy Table

Table 2 reports per-task exact-match accuracy for four models across three tokenizer views. Each cell is the hit rate for one task under one tokenizer in a fixed run configuration.

5.4 Observed Failure Modes

Case-level annotations show recurring miss categories:

- invalid output format,
- operation mismatch,

Table 2. Task-wise exact-match accuracy across models and tokenizer views. T1–T4 are label tasks; T5–T8 are sequence-transformation tasks.

Model	Tokenizer	T1	T2	T3	T4	T5	T6	T7	T8
GPT-5-mini	note_level	0.95	0.90	0.90	0.90	0.75	0.65	0.70	0.70
GPT-5-mini	midilike	0.95	0.90	0.85	0.90	0.70	0.60	0.65	0.65
GPT-5-mini	remilike	0.95	0.90	0.85	0.90	0.70	0.65	0.65	0.70
Qwen2.5-7B	note_level	0.70	0.65	0.55	0.60	0.45	0.35	0.40	0.45
Qwen2.5-7B	midilike	0.70	0.60	0.50	0.55	0.35	0.25	0.30	0.35
Qwen2.5-7B	remilike	0.70	0.60	0.55	0.55	0.40	0.30	0.35	0.40
Qwen3-4B	note_level	0.75	0.65	0.60	0.65	0.50	0.40	0.45	0.45
Qwen3-4B	midilike	0.75	0.65	0.55	0.60	0.40	0.30	0.35	0.40
Qwen3-4B	remilike	0.75	0.65	0.60	0.60	0.45	0.35	0.40	0.45
Qwen2.5-3B	note_level	0.60	0.50	0.40	0.45	0.30	0.20	0.25	0.30
Qwen2.5-3B	midilike	0.55	0.45	0.35	0.40	0.20	0.10	0.15	0.20
Qwen2.5-3B	remilike	0.55	0.45	0.40	0.40	0.25	0.15	0.20	0.25

- label or sequence mismatch against the exact ground truth.

These failures are inspectable through stored per-case artifacts, enabling direct debugging of prompt or decoding strategies.

6. DISCUSSION

6.1 What This Benchmark Measures

The benchmark measures structured symbolic reasoning under constraints, not open-ended compositional quality. It is useful for controlled comparison of model behavior on formal theory operations.

6.2 Musicological Interpretation

The benchmark is intended to support a musicological question, not only an engineering one. Intervals, chords, and harmonic function are treated as analytical categories. Transposition, inversion, and retrograde are treated as transformational operations. Voice-leading violations are treated as normative judgments grounded in common-practice theory. In this sense, the benchmark operationalizes concepts that already matter in analytical discourse.

The optional explanation layer extends this framing. A model can predict the same correct label but still fail to provide a convincing analytical explanation. For example, a system may label a chord as `dominant` without being able to articulate that it functions as V in the context of a key and projects tension toward the tonic. The benchmark therefore separates structural correctness from explanatory adequacy and makes that gap observable.

6.3 Current Limits

The release is intentionally narrow: major-mode harmonic function, reduced chord vocabulary, and two-voice voice-leading. It does not yet evaluate richer tonal contexts, counterpoint regimes, or stylistic interpretation.

6.4 Next Steps

Priority extensions include expanded harmonic contexts, more complex voice-leading settings, constrained decoding at token level, and larger-scale comparative runs with stable statistical reporting.

7. CONCLUSION

We introduced a symbolic benchmark for music theory reasoning and computational musicology that combines deterministic case generation, synchronized tokenizer views, strict prompt contracts, automatic scoring, and an optional analytical explanation layer. By design, the benchmark emphasizes reliability and inspectability over maximal theory breadth. Its central value is not only to compare models, but also to clarify whether formal symbolic correctness can stand in for musical understanding.

8. AI USAGE STATEMENT

Coding and drafting support tools were used for prototyping documentation, scripts, and benchmark interfaces. All benchmark definitions, evaluation logic, and manuscript content should be manually reviewed by the authors before submission.

9. REFERENCES

[1] G. Peeters, Z. Rafii, M. Fuentes, Z. Duan, E. Benetos, J. Nam, and Y. Mitsufuji, "Twenty-five years of mir research: Achievements, practices, evaluations, and future challenges," in *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2025.

[2] Z. Zhou, Y. Wu, Z. Wu, X. Zhang, R. Yuan, Y. Ma, L. Wang, E. Benetos, W. Xue, and Y. Guo, "Can llms "reason" in music? an evaluation of llms' capability of music understanding and generation," *arXiv preprint arXiv:2407.21531*, 2024.

[3] Y. Huang and Y. Yang, "Pop music transformer: Generating music with rhythm and harmony," in *Proceedings of the 28th ACM International Conference on Multimedia*, 2020, pp. 1180–1189.

[4] Y. Huang, H. Liu, Y. Liu, and Y. Yang, "Musicbert: Symbolic music understanding with large-scale pre-training," *arXiv preprint arXiv:2106.05630*, 2021.

[5] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. A. Hudson *et al.*, "Holistic evaluation of language models," *Transactions on Machine Learning Research*, 2023.

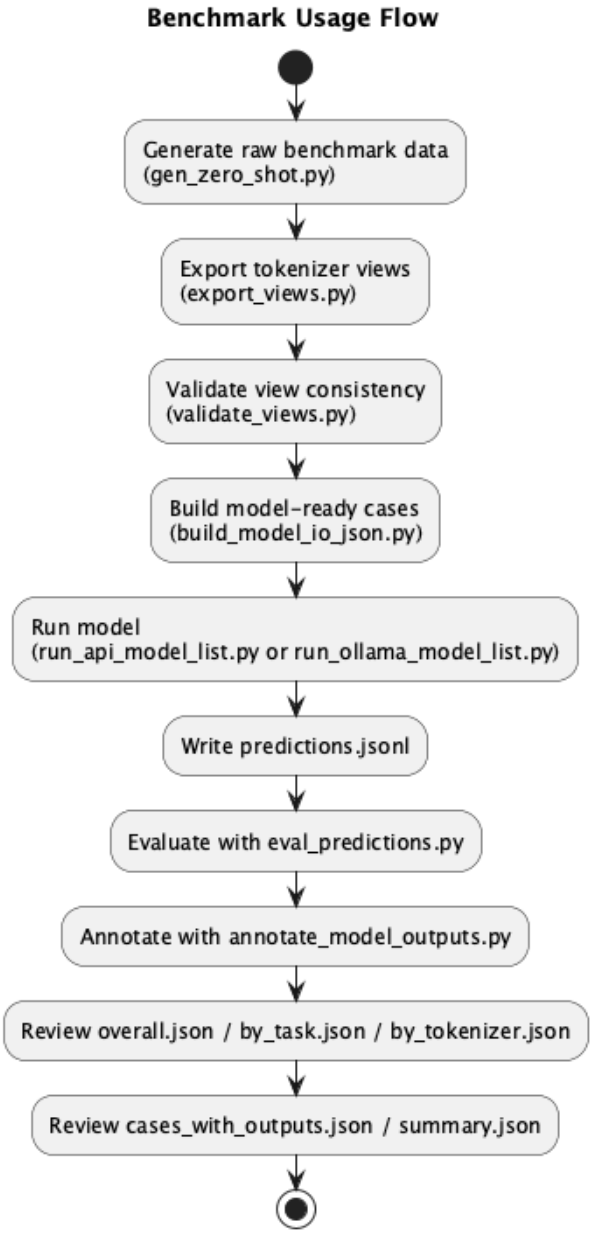

Figure 3. Usage flow in the benchmark suite: generation, execution, and summarized outputs.